

Trabajo de SED

Micros

Grupo 10

Participantes

Guillermo Rosales Pérez	Nº 56070
Lorenzo Hidalgo Anero	Nº 56415
Matías López Viagel	Nº 56444

Índice

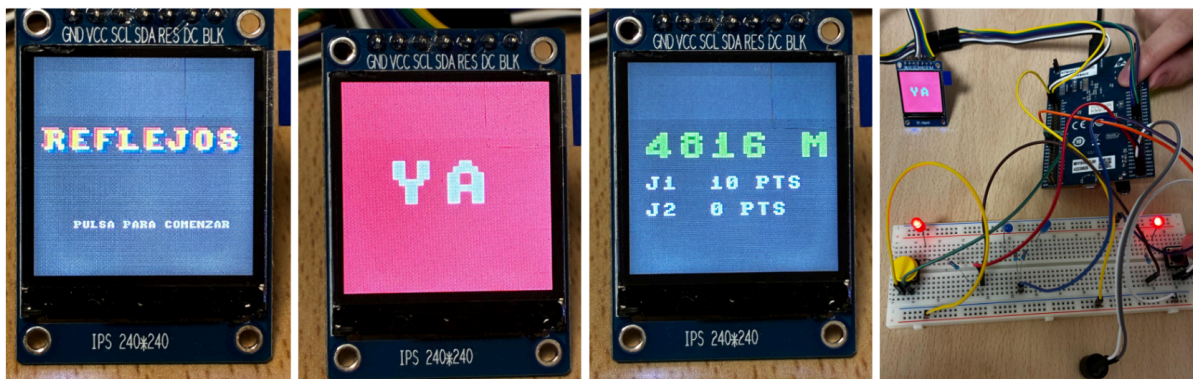
1. Introducción y objetivos.....	3
1.1. Tarea a realizar.....	3
1.2. Objetivos.....	3
2. Descripción del proyecto.....	4
2.1. Funcionamiento.....	4
2.2 Modos de juego.....	4
2.2.1. Cuenta Atrás.....	4
2.2.2. Mejor Resultado.....	5
2.3 Componentes.....	5
3. Código.....	5
3.1 Estructura.....	5
3.2 Bloques.....	9
3.2.1 Bloques .cpp.....	9
3.2.1. 1. Captura_boton.cpp.....	9
3.2.1. 2. Maquina Menu.cpp.....	12
3.2.1. 3. Lógica_Juego.cpp.....	17
3.2.1. 4. Estilo_pantalla.cpp.....	23
3.2.1. 6. Driver_pantalla.cpp.....	29
3.2.1. 6. Dibujar_juego.cpp.....	31
3.2.1. 7. Main.cpp.....	33
3.2.2. Bloques .h.....	42
3.2.2. 1. Captura_boton.h.....	42
3.2.2. 2. Maquina_menu.h.....	42
3.2.2. 3. Lógica_Juego.h.....	43
3.2.2. 4. Estilo_pantalla.h.....	44
3.2.2. 5. Driver_pantalla.h.....	44
3.2.2. 6. Dibujar_juego.h.....	45
3.2.2. 7. Main.h.....	45
3.2.2. 8. GestorPuntuacion.h.....	47
3.2.2. 9. Definiciones.h.....	48
4. Github.....	49
5. Conclusión.....	49

1. Introducción y objetivos

1.1. Tarea a realizar

Como trabajo de la asignatura de Sistemas Electrónicos Digitales hemos decidido hacer un juego de "Reflejos al Límite" en un microcontrolador STM32F411. El trabajo elegido era una de las propuestas presentadas por los profesores de la asignatura, dado que al no tener experiencia previa en el manejo de las microcontroladores hemos preferido trabajar con lo que nos recomendaban. El enunciado del trabajo es el siguiente:

"Este juego pone a prueba la rapidez de reacción de los jugadores. Cada participante dispone de un botón y un LED propio. En cada ronda, el sistema enciende aleatoriamente uno de los LEDs; el jugador correspondiente debe pulsar su botón lo más rápido posible. Si reacciona a tiempo, gana puntos; si se adelanta o se retrasa, el sistema emite una señal de error. El microcontrolador gestiona los tiempos de encendido, mide el intervalo de reacción y actualiza los puntajes en pantalla. ".



1.2. Objetivos

El principal propósito del trabajo es el diseño de un juego competitivo capaz de ser jugado por dos personas de manera simultánea. Para llevar esto a cabo los principales objetivos son:

- Sincronizar entradas asíncronas.
- Inicializar el juego mediante un botón
- Mostrar con un display información del juego.
- Identificar qué jugador pulsa antes.
- Calcular la puntuación a raíz del tiempo tardado en pulsar.

Nosotros como objetivo extra hemos decidido añadir:

- Selección entre varios modos de juego.
- Añadir modo 1 Jugador y 2 Jugadores.
- Añadir 3 modos de dificultad.
- Implementar una pantalla OLED en vez de usar un display
- Mostrar por pantalla toda la información real que tendría un videojuego.

2. Descripción del proyecto

2.1. Funcionamiento

El funcionamiento del juego consiste en lo siguiente. Una vez iniciado el juego aparecerá una pantalla de espera, en la cuál tendremos que presionar el botón del microcontrolador para comenzar la primera ronda. Una vez presionado el botón se cargará un valor pseudoaleatorio en el temporizador y cuando dicho temporizador llegue a 0 se encenderán los leds y se mostrará por pantalla un mensaje, ambos indicando que cada jugador debe de presionar su botón. El jugador que haya presionado más rápido su botón será el que gane más puntos, acercándole así a la victoria.

En nuestro caso al tener 2 modos de juego diferentes el número de rondas varía. En un modo de juego se juega a 3 rondas y en el otro a 2 rondas. El número de rondas elegido en cada juego es el número mínimo de rondas necesarias en cada caso para poder decidir un ganador claro y poder mostrar la funcionalidad del programa. En caso de querer añadir más rondas se podría mediante el cambio de una variable en el código, permitiendo así hacer juegos de 5, 8, 10 o más rondas si se deseara.

2.2 Modos de juego

2.2.1. Cuenta Atrás

El primer modo de juego es el denominado "Cuenta Atrás". En este modo de juego competirán ambos jugadores para ser el primero en pulsar el botón, utilizando las normas más básicas del juego base "Reflejos Rápidos". Este modo de juego se juega a 3 rondas y tiene la particularidad de que sólo puntúa el jugador más rápido, esto significa que el código solo permitirá al jugador más rápido puntuar. Cada vez que se gana una ronda el jugador gana 100 puntos, llegando a ser la puntuación máxima por partida 300 puntos y la mínima 0 puntos.

2.2.2. Mejor Resultado

En este segundo modo de juego se aplica el mismo concepto que en el juego base "Reflejos Rápidos", según el cual, el primer jugador que pulse gana la ronda. En este modo de juego ambos jugadores pueden pulsar su botón una vez por ronda sin importar que el otro jugador lo haya pulsado antes. A partir del tiempo que cada jugador tarda en pulsar su botón se calcula según una tabla de puntuaciones el puntaje obtenido en esa ronda, siendo el puntaje mínimo por ronda de 10 puntos y el máximo de 100 puntos. En este modo de juego se competirá a 2 rondas debido a que siempre existe la posibilidad de que ambos jugadores puedan empatar muy poco probable que ambos jugadores lleguen a sacar el mismo puntaje final, y menos después de solo 2 rondas.

2.3 Componentes

- OLED TFT 7768
- 1 BUZZER
- LEDS DE COLORES AZUL Y ROJO

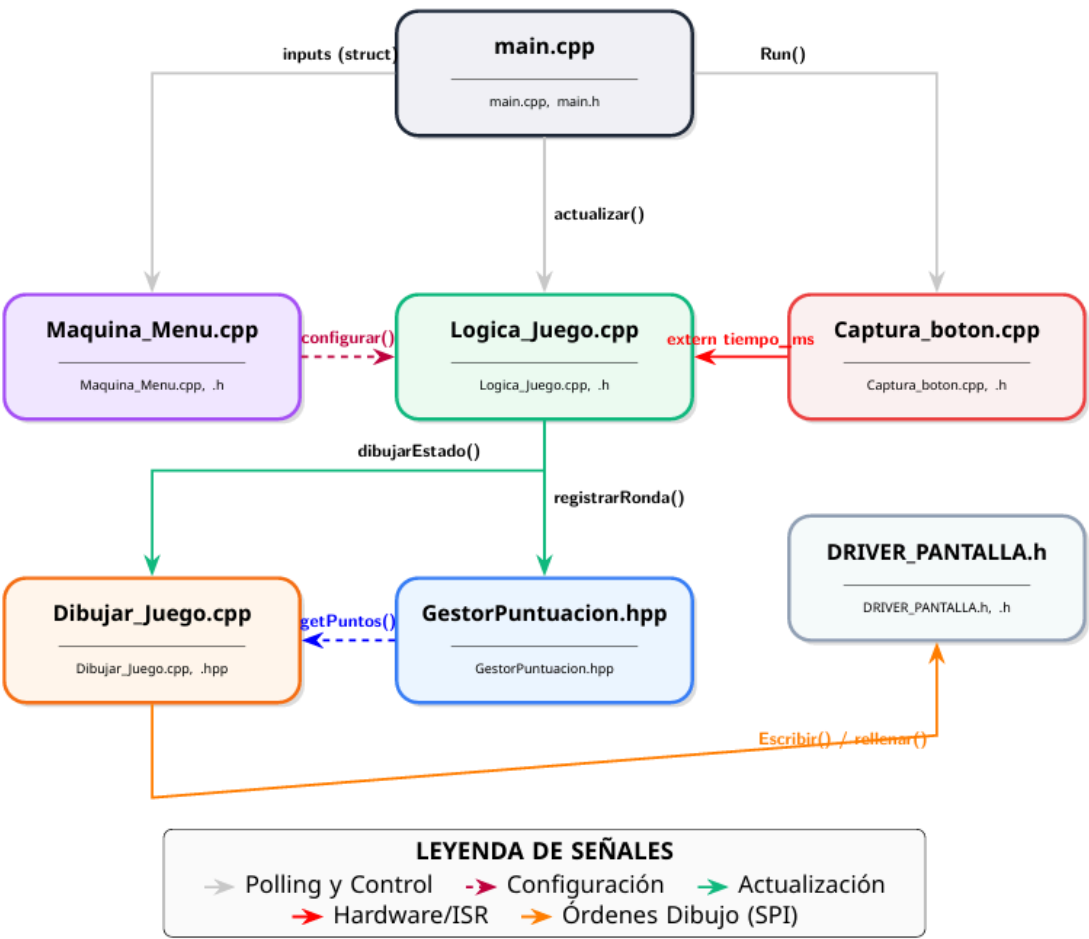
3. Código

3.1 Estructura

El proyecto se ha diseñado siguiendo una estructura de Programación Orientada a Objetos (POO). Siguiendo un poco con lo aprendido en FPGAS se ha procurado seguir con ese alto grado de modularidad que nos permite alcanzar y a la posibilidad de encapsular el código. Esto facilita, tanto la división del trabajo entre los componentes del equipo a la hora de trabajar, como mayor sencillez a la hora de estructurar el programa, detectar errores y diseñar mejor la lógica de los procesos.

Utilizar este modelo de programación nos permite modificar partes diferenciadas del código sin llegar a afectar en gran medida al resto del proyecto. Gracias a ello se ha conseguido detectar dónde se producían los fallos, el poder trabajar con un bloque aislado y solo modificar dicho bloque.

ARQUITECTURA FUNCIONAL DEL SISTEMA



En nuestro proyecto hemos decidido utilizar 2 máquinas de estado independientes. La primera máquina ([Maq_Estados](#)) se encarga de controlar los modos de juego, funcionando a modo de menú dónde puedes elegir la dificultad, el número de jugadores o el modo de juego. La segunda máquina de estado ([Logica_Juego](#)) se enfoca en el propio control del juego, realizando las llamadas a los distintos estados de manera que cada vez que se realice una ronda o se acabe el juego, se cambie al estado siguiente. Por último existe algo análogo a una máquina de estados para el control de lo que se imprime por la OLED bajo el control de maquina_Menu, éste modulo es [Dibujar_Juego](#), que podría describirse como una máquina esclava a otra máquina de estados, ésta cambia lo que se muestra por la OLED alternando entre poner pantallas de carga, las puntuaciones de la ronda o la pantalla de inicio de partida.

A parte para el control de la pantalla tenemos 2 bloques especializados los cuales son [DRIVER_PANTALLA](#) y [ESTILO_PANTALLA](#). El primero se centra en la comunicación de bajo nivel con el hardware, gestionando el protocolo SPI y el envío de datos crudos al controlador de la pantalla. Por su parte, el segundo bloque utiliza estas funciones básicas para ofrecer una capa más manejable por el programador, de alto nivel, permitiendo dibujar textos directamente sin preocuparnos por la manipulación individual de los píxeles.

También contamos con un bloque dedicado a la interpretación de las capturas de los botones. Dicho bloque se llama [Captura_boton](#) y su principal función es identificar que botón se ha pulsado y según se haya presionado uno u otro activar o desactivar distintos valores, los cuales nos ayudarán a identificar al ganador de cada ronda entre otras cosas.

Por último nos queda mencionar los bloques de [GestorPuntuacion](#) y [Definiciones](#), los cuales son los únicos bloques que solo cuentan con un archivo .h. Esto es debido a que funcionan a modo de Interfaz, permitiendo en el caso de [GestorPuntuacion](#) que se pueda acceder en todo momento a la operación de calcular los puntos y en el caso de [Definiciones](#) permite a todos los bloques poder conocer las variables de estados de juego y las distintas flags utilizadas en las máquinas de estado.

- >  Captura_boton.h
 - >  Definiciones.h
 - >  Dibujar_Juego.hpp
 - >  DRIVER_PANTALLA.h
 - >  ESTILO_PANTALLA.h
 - >  GestorPuntuacion.hpp
 - >  Logica_Juego.h
 - >  main.h
 - >  Maquina_Menu.h
 - >  stm32f4xx_hal_conf.h
 - >  stm32f4xx_it.h
 - >  Captura_boton.cpp
 - >  Dibujar_Juego.cpp
 - >  DRIVER_PANTALLA.cpp
 - >  ESTILO_PANTALLA.cpp
 - >  Logica_Juego.cpp
 - >  main.cpp
 - >  Maquina_Menu.cpp
 - >  stm32f4xx_hal_msp.c
 - >  stm32f4xx_it.c
 - >  syscalls.c
 - >  sysmem.c
 - >  system_stm32f4xx.c

3.2 Bloques

3.2.1 Bloques .cpp

3.2.1. 1. Captura_boton.cpp

Captura botón se encarga de controlar y detectar cuando los jugadores activan los pulsadores y guarda el tiempo de reacción de estos. A su vez, también diferencia quién de los jugadores ha pulsado primero para así poder dar los puntos correspondientes. Al comienzo se activa el ADC, que lee un pin analógico capturando el ruido, esto se usa para así poder generar números aleatorios en la entrada y no sea predecible cuando va a encender los LEDs, esta variable se guarda en [adc_ruido](#). A su vez, al principio también se reinician las variables internas para que así cada ronda sea idéntica. Una vez se enciende el temporizador [TIM2](#), el cual lo hace siempre y cuando no haya habido pulsaciones previas para así evitar las trampas, empieza su cuenta. Las pulsaciones se detectan mediante interrupciones, según el modo de juego se guardará el tiempo de la primera pulsación válida de un jugador o de los dos. Si la interrupción es correcta, se reinicia el temporizador y se asigna el ganador.

CÓDIGO

```
#include "Captura_boton.h"
#include "main.h"
#include <stdlib.h>
#include <stdint.h>
#include "stm32f4xx_hal.h"
// Referencias a handles de hardware proporcionados por STM32Cube.
// Por esto:
extern "C" {
    extern TIM_HandleTypeDef htim2;
    extern ADC_HandleTypeDef hadc1;
}
// Variables globales para almacenar el tiempo en milisegundos y
// el valor de ruido de ADC usados para la generación de números aleatorios.
// Estas variables se actualizan en las callbacks de HAL.
volatile uint32_t tiempo_ms = 0;
volatile uint16_t adc_ruido = 0;
// Variables internas para registrar el momento de la pulsación y el jugador ganador.
static volatile uint8_t boton_pulsado = 0;
static volatile uint32_t resultado_final = 0;
static volatile uint32_t resultado_final_1 = 0;
static volatile uint32_t resultado_final_2 = 0;
static uint8_t ganador_local = 0;
extern volatile bool irq_pa0_pressed; // Traemos el flag del main
extern volatile bool Presiona_Jugador_1;
extern volatile bool Presiona_Jugador_2;
// Indicador de si el temporizador está en marcha. Esto evita reiniciar
// accidentalmente el conteo en cada llamada a Run.
static bool temporizador_activo = false;
// Callback de HAL para el temporizador. Se llama cada vez que el temporizador
// genera una interrupción (configurado a 1 kHz para incrementar 1 ms).
extern "C" void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        tiempo_ms++;
    }
}
// Callback de HAL para la conversión del ADC. Captura ruido para usarlo
// como semilla de números aleatorios.
extern "C" void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc) {
    if (hadc->Instance == ADC1) {
        adc_ruido = static_cast<uint16_t>(HAL_ADC_GetValue(hadc));
    }
}
// Inicializa la captura del botón: arranca el ADC en modo interrupción y
// configura la semilla del generador aleatorio con el ruido inicial.
void Captura_Boton_Init(void) {
    HAL_ADC_Start_IT(&hadc1);
    // Permite que el ADC realice al menos una conversión antes de usar su ruido.
    HAL_Delay(10);
}
```

```

    srand(adc_ruido);
    boton_pulsado = 0;
    resultado_final = 0;
    ganador_local = 0;
    temporizador_activo = false;
    tiempo_ms = 0;
}
void Captura_Boton_Run(void) {
    if (!temporizador_activo) {
        // SEGURIDAD: Solo arranca si el botón está SUELTO para evitar trampas
        if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET) return;
        tiempo_ms = 0;
        __HAL_TIM_SET_COUNTER(&htim2, 0);
        HAL_TIM_Base_Start_IT(&htim2);
        temporizador_activo = true;
        boton_pulsado = 0;
        resultado_final = 0;
        irq_pa0_pressed = false; // Limpiamos basuras previas
    }
    if (boton_pulsado == 0) {
        // Prioridad J1: Detectamos si la interrupción ocurrió durante la ronda
        if (irq_pa0_pressed) {
            resultado_final = tiempo_ms;
            boton_pulsado = 1;
            ganador_local = 1;
            HAL_TIM_Base_Stop_IT(&htim2);
            temporizador_activo = false;
            irq_pa0_pressed = false;
            HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_SET);
        }
        // J2: Polling normal en PA10
        else if (Presiona_Jugador_1) {
            resultado_final_1 = tiempo_ms;
            boton_pulsado = 1;
            ganador_local = 1;
            HAL_TIM_Base_Stop_IT(&htim2);
            temporizador_activo = false;
            Presiona_Jugador_1 = false;
            HAL_GPIO_WritePin(LED_ROJO_GPIO_Port, LED_ROJO_Pin, GPIO_PIN_SET);
        }
        // J2: Polling normal en PA10
        else if (Presiona_Jugador_2) {
            resultado_final_2 = tiempo_ms;
            boton_pulsado = 1;
            ganador_local = 2;
            HAL_TIM_Base_Stop_IT(&htim2);
            temporizador_activo = false;
            Presiona_Jugador_2 = false;
            HAL_GPIO_WritePin(LED_ROJO_GPIO_Port, LED_ROJO_Pin, GPIO_PIN_SET);
        }
    }
}

```

```

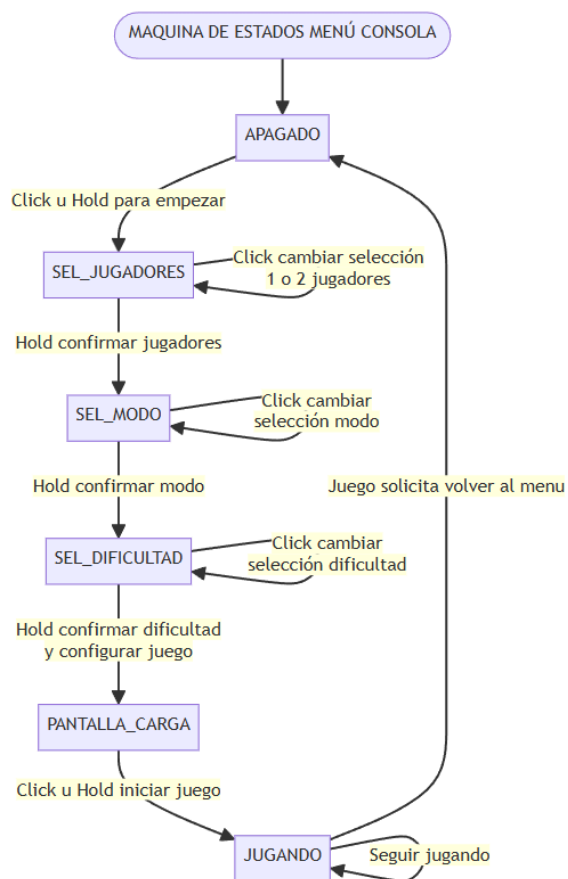
}

// Funciones auxiliares para consultar el resultado de la captura.
// Devuelven el tiempo registrado y el jugador que ganó.
uint32_t Captura_GetTiempoFinal() {
    return resultado_final;
}
uint8_t Captura_GetGanador() {
    return ganador_local;
}
}

```

3.2.1. 2. Maquina Menu.cpp

Maquina Menu es un módulo que funciona como una máquina de estados que se encarga de gestionar el menú principal del sistema. Es decir, a diferencia de Lógica_Juego el cual es una máquina de estados que se encarga de gestionar la lógica del propio juego, Maquina Menu permite al usuario configurar el juego y transicionar entre el menú y la partida.



Este módulo también se encarga de representar gráficamente en que estado nos encontramos, dibujando los textos, colores y efectos visuales en pantalla para indicar el estado actual y la opción seleccionada. Una vez completada la configuración, envía los parámetros elegidos al módulo de Lógica_Juego y así comenzar la partida. Durante el tiempo que se está en Logica_Juego, Maquina Menu está pendiente de la señal que pueda recibir para así comenzar de nuevo su flujo.

[illegible]

```

    if (inputs.btn_PA0_Click) {
        // logica de alternancia simple
        if (_selJugadores == OpcionJugadores::UN_JUGADOR)
            _selJugadores = OpcionJugadores::FANTASMA_1;
        else if (_selJugadores == OpcionJugadores::FANTASMA_1)
            _selJugadores = OpcionJugadores::DOS_JUGADORES;
        else if (_selJugadores == OpcionJugadores::DOS_JUGADORES)
            _selJugadores = OpcionJugadores::FANTASMA_2;
        else if (_selJugadores == OpcionJugadores::FANTASMA_2)
            _selJugadores = OpcionJugadores::UN_JUGADOR;
        dibujarMenuJugadores();
    }
    if (inputs.btn_PA0_Hold) { //si mantenemos el boton cambiamos de
estado
        _selModo = OpcionModo::CUENTA_ATRAS; // dejamos seleccionado la
primera opcion
        _estadoActual = EstadoMenu::SEL_MOD0;
        _pantalla->rellenarPantalla(0x0005);
        dibujarMenuModo();
        const_cast<Entradas&>(inputs).btn_PA0_Hold = false;
    }
    break;
case EstadoMenu::SEL_MOD0:
    if (inputs.btn_PA0_Click) { //si le damos click cambiamos de
seleccion
        if (_selModo == OpcionModo::CUENTA_ATRAS)
            _selModo = OpcionModo::FANTASMA_3;
        else if (_selModo == OpcionModo::FANTASMA_3)
            _selModo = OpcionModo::MEJOR_RESULTADO;
        else if (_selModo == OpcionModo::MEJOR_RESULTADO)
            _selModo = OpcionModo::FANTASMA_4;
        else if (_selModo == OpcionModo::FANTASMA_4)
            _selModo = OpcionModo::CUENTA_ATRAS;
        dibujarMenuModo();
    }
    if (inputs.btn_PA0_Hold) { // si holdeamos se queda guardada la
configuracion anterior del selector
        _selDificultad = OpcionDificultad::FACIL; // dejamos el nivel ya
seleccionado
        _estadoActual = EstadoMenu::SEL_DIFICULTAD; //cambiamos de estado
        _pantalla->rellenarPantalla(0x0005);
        dibujarMenuDificultad();
        const_cast<Entradas&>(inputs).btn_PA0_Hold = false;
    }
    break;
case EstadoMenu::SEL_DIFICULTAD:
    if (inputs.btn_PA0_Click) {
        if (_selDificultad == OpcionDificultad::FACIL)
            _selDificultad = OpcionDificultad::FANTASMA_5;
        else if (_selDificultad == OpcionDificultad::FANTASMA_5)
            _selDificultad = OpcionDificultad::MEDIO;
    }

```

[illegible]

```

        _inicioDibujado = true;
        _tickParpadeo = HAL_GetTick();
    }
    // TEXTO PEQUEÑO PARPADEANTE
    if ((HAL_GetTick() - _tickParpadeo) > 500) {
        _tickParpadeo = HAL_GetTick();
        _textoVisible = !_textoVisible;
        _pantalla->setEscala(1); // LETRA PEQUEÑA
        if (_textoVisible)
            _pantalla->Escribir(44, 180, "Pulsa para comenzar", 0xFFFF, 0, 0);
        else
            _pantalla->Escribir(44, 180, "Pulsa para comenzar", 0x0000, 0, 0);
        _pantalla->setEscala(2); // Restaurar escala
    }
}

void Maquina_Menu::dibujarMenuJugadores() {
    _pantalla->setEscala(2);
    // Colores y Flechas
    uint16_t c1 = (_selJugadores == OpcionJugadores::UN_JUGADOR) ? 0xFFE0 :
0xFFFF;
    uint16_t c2 = (_selJugadores == OpcionJugadores::DOS_JUGADORES) ? 0xFFE0 :
0xFFFF;
    const char* f1 = (_selJugadores == OpcionJugadores::UN_JUGADOR) ? ">" : " ";
    const char* f2 = (_selJugadores == OpcionJugadores::DOS_JUGADORES) ? ">" : "
";
    _pantalla->Escribir(20, 20, "JUGADORES", 0xFFFF, 0, 0);
    _pantalla->Escribir(20, 60, f1, c1, 0, 0);
    _pantalla->Escribir(40, 60, "1 JUGADOR", c1, 0, 0);
    _pantalla->Escribir(20, 90, f2, c2, 0, 0);
    _pantalla->Escribir(40, 90, "2 JUGADORES", c2, 0, 0);
    _pantalla->setEscala(1);
    _pantalla->Escribir(10, 210, "Click:Cambiar Hold:Ok", 0x07E0, 0,0);
}

void Maquina_Menu::dibujarMenuModo() {
    _pantalla->setEscala(2);
    uint16_t c1 = (_selModo == OpcionModo::CUENTA_ATRAS) ? 0xFFE0 : 0xFFFF;
    uint16_t c2 = (_selModo == OpcionModo::MEJOR_RESULTADO) ? 0xFFE0 : 0xFFFF;
    const char* f1 = (_selModo == OpcionModo::CUENTA_ATRAS) ? ">" : " ";
    const char* f2 = (_selModo == OpcionModo::MEJOR_RESULTADO) ? ">" : " ";
    _pantalla->Escribir(20, 20, "MODO JUEGO", 0xFFFF, 0, 0);
    _pantalla->Escribir(10, 60, f1, c1, 0, 0);
    _pantalla->Escribir(30, 60, "CUENTA ATRAS", c1, 0, 0);
    _pantalla->Escribir(10, 90, f2, c2, 0, 0);
    _pantalla->Escribir(30, 90, "MEJOR TIEMPO", c2, 0, 0);
    _pantalla->setEscala(1);
    _pantalla->Escribir(10, 210, "Click:Cambiar Hold:Ok", 0x07E0, 0,0);
}

void Maquina_Menu::dibujarMenuDificultad() {
    _pantalla->setEscala(2);
    uint16_t c1 = (_selDificultad == OpcionDificultad::FACIL) ? 0xFFE0 :
0xFFFF;

```



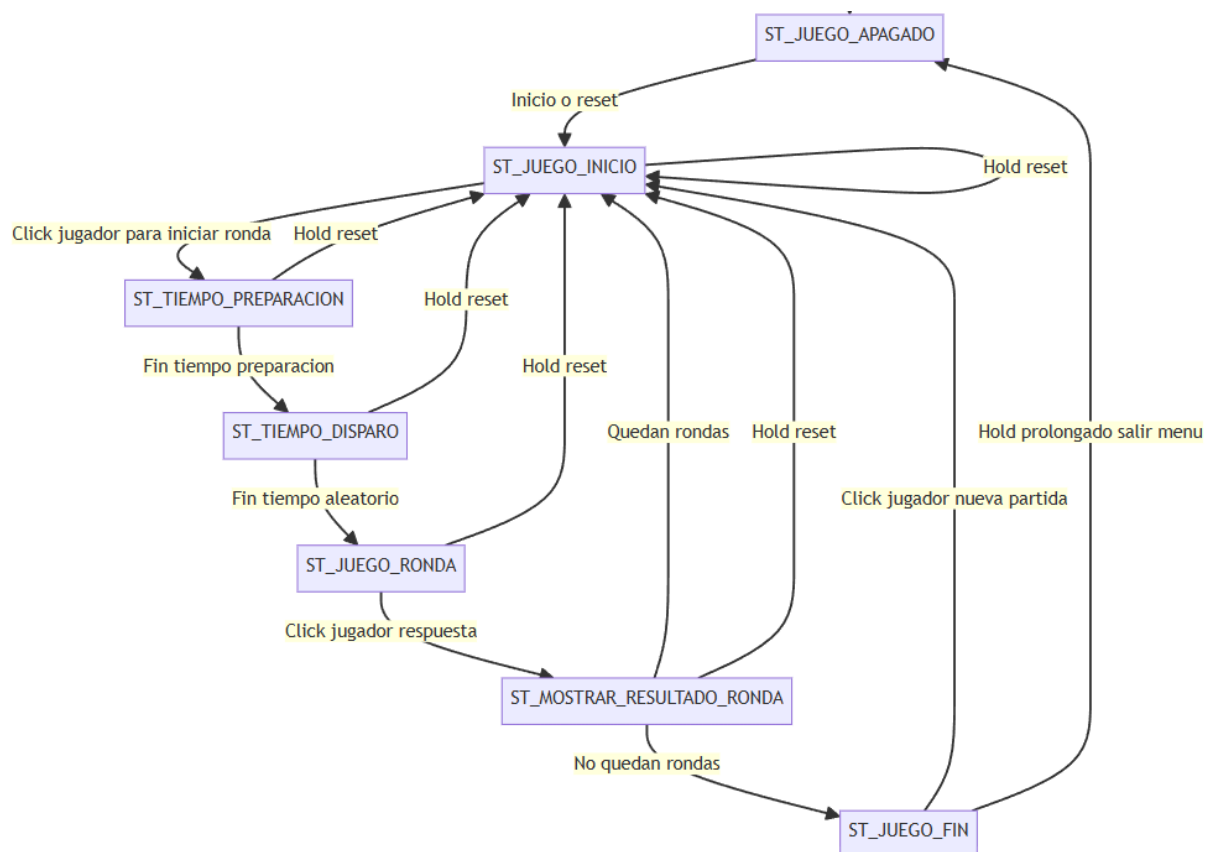
```

uint16_t c2 = (_selDificultad == OpcionDificultad::MEDIO) ? 0xFFE0 :
0xFFFF;
uint16_t c3 = (_selDificultad == OpcionDificultad::DIFICIL) ? 0xFFE0 :
0xFFFF;
const char* f1 = (_selDificultad == OpcionDificultad::FACIL) ? ">" : " ";
const char* f2 = (_selDificultad == OpcionDificultad::MEDIO) ? ">" : " ";
const char* f3 = (_selDificultad == OpcionDificultad::DIFICIL) ? ">" : " ";
_pantalla->Escribir(20, 20, "DIFICULTAD", 0xFFFF, 0, 0);
_pantalla->Escribir(20, 60, f1, c1, 0, 0);
_pantalla->Escribir(40, 60, "FACIL", c1, 0, 0);
_pantalla->Escribir(20, 90, f2, c2, 0, 0);
_pantalla->Escribir(40, 90, "MEDIO", c2, 0, 0);
_pantalla->Escribir(20, 120, f3, c3, 0, 0);
_pantalla->Escribir(40, 120, "DIFICIL", c3, 0, 0);
_pantalla->setEscala(1);
_pantalla->Escribir(10, 210, "Click:Cambiar Hold:Ok", 0x07E0, 0,0);
}
void Maquina_Menu::dibujarCarga() {
_pantalla->setEscala(2);
_pantalla->Escribir(40, 60, "LISTO?", 0xFFFF, 0, 0);
_pantalla->Escribir(30, 110, "> EMPEZAR <", 0x07E0, 0xF800, 0);
_pantalla->setEscala(1);
_pantalla->Escribir(50, 200, "HOLD TO START", 0xFFFF, 0, 0);
}

```

3.2.1. 3. Lógica_Juego.cpp

Lógica_juego es el núcleo una vez se ha llegado al estado de juego. Este módulo funciona como una máquina de estados que gestiona las distintas fases del juego. Se coordina con el resto de módulos ya que recibe información de ellos a la vez que envía información nueva.



Todos los estados se gestionan en un switch, que en función del **estadoActual** hace una cosa u otra. En cada estado se comprueban las entradas del usuario y el tiempo transcurrido para decidir cuándo se produce una transición al siguiente estado. En los estados previos a empezar el juego va actualizando la pantalla OLED para así poder avisar al usuario cuando va a comenzar el juego.

Una vez finalizada la ronda, el sistema pasa a un estado en el que muestra el resultado durante un tiempo determinado, antes de iniciar una nueva ronda o finalizar el juego si se ha alcanzado el número máximo de rondas. En el estado final, el usuario puede reiniciar la partida o salir al menú principal mediante distintas acciones sobre el botón.

```
#include "Logica_Juego.h"
#include <stdlib.h>
#include <stdio.h>
// Referencia a variables globales del hardware (definidas en main.c)
```

```

// Nota: "tiempo_ms" se ha dejado de utilizar para medir el tiempo de reacción.
En su lugar
// se emplea HAL_GetTick(), que devuelve el número de milisegundos desde el
arranque del sistema.
extern volatile uint16_t adc_ruido;
Logica_Juego::Logica_Juego(Estilo_Pantalla* pantalla)
    : _pantalla(pantalla),
      _dibujante(pantalla) {
    // Inicializamos la máquina de estados y los registros de puntuación.
    resetearJuegoCompleto();
}

void Logica_Juego::actualizar(const Entradas& inputs) {
    uint32_t Tiempo = HAL_GetTick();
    // 1. DESPERTAR / REINICIO
    if (_estadoActual == EstadoJuego::ST_JUEGO_APAGADO) {
        _puntuacion.resetear();
        _estadoActual = EstadoJuego::ST_JUEGO_INICIO;
        _redibujar = true;
        _solicitudSalidaMenu = false; // Aseguramos que empiece limpia
        return;
    }
    // HOLD GLOBAL: Reinicia el juego en cualquier ronda
    // RESTRICCIÓN: Solo actúa si NO estamos en el menú de fin de juego
    if (inputs.btn_PA0_Hold && _estadoActual != EstadoJuego::ST_JUEGO_FIN) {
        _puntuacion.resetear();
        _estadoActual = EstadoJuego::ST_JUEGO_INICIO;
        _redibujar = true;
        HAL_GPIO_WritePin(LED_AZUL_GPIO_Port, LED_AZUL_Pin,
GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LED_ROJO_GPIO_Port, LED_ROJO_Pin,
GPIO_PIN_RESET);
        return; // Salimos para no procesar el switch en este ciclo
    }
    //este hold esta fuera de los switch porque quiero que el hold me permita
reiniciar el juego en cualquier ronda pero en ese ultimo menu de finjuego quiero
que me saque al menu_maquina (la consola)
    // 2. MÁQUINA DE ESTADOS
    switch (_estadoActual) {
        case EstadoJuego::ST_JUEGO_INICIO:
            if (inputs.btn_PA0_Click) {
                _solicitudSalidaMenu = false;
                _tickInicio = Tiempo;
                _estadoActual = EstadoJuego::ST_TIEMPO_PREPARACION;
                _redibujar = true; // Dispara el dibujo del fondo azul
                HAL_GPIO_WritePin(LED_AZUL_GPIO_Port, LED_AZUL_Pin,
GPIO_PIN_SET);
            }
            break;
        case EstadoJuego::ST_TIEMPO_PREPARACION:
            if (Tiempo - _tickInicio >= 1500) {
                _tiempoAleatorio = obtenerTiempoEspera();
            }
    }
}

```

```

        _tickInicio = Tiempo;
        _estadoActual = EstadoJuego::ST_TIEMPO_DISPARO;
        _redibujar = true; // Fondo negro
        HAL_GPIO_WritePin(LED_AZUL_GPIO_Port, LED_AZUL_Pin,
GPIO_PIN_RESET);
    }
    break;
    case EstadoJuego::ST_TIEMPO_DISPARO:
        if (Tiempo - _tickInicio >= _tiempoAleatorio) {
            _tickInicioDisparo = Tiempo;
            _estadoActual = EstadoJuego::ST_JUEGO_RONDA;
            _redibujar = true; // Fondo rojo + "YA!"
            HAL_GPIO_WritePin(LED_ROJO_GPIO_Port, LED_ROJO_Pin,
GPIO_PIN_SET);
        }
        break;
    case EstadoJuego::ST_JUEGO_RONDA:
        //Solo detecta el botón del jugador 1.
        if (_confJugadores == OpcionJugadores::UN_JUGADOR ){
            if (inputs.Boton_1) {
                _tiempoReaccion_1 = Tiempo - _tickInicioDisparo;
                _puntuacion.registrarRonda(_tiempoReaccion_1, _confModo, 1);
                _tickInicio = Tiempo;
                _estadoActual = EstadoJuego::ST_MOSTRAR_RESULTADO_RONDA;
                _redibujar = true;
                // --- NUEVO: Apagar LED y feedback sonoro corto ---
                Buzzer_Beep(50); // Pitido muy corto de 50ms
                HAL_GPIO_WritePin(LED_ROJO_GPIO_Port, LED_ROJO_Pin,
GPIO_PIN_RESET);
            }
        }

        // MODO PARA 2 JUGADORES CON MEJOR_RESULTADO
        if (_confJugadores == OpcionJugadores::DOS_JUGADORES && _confModo ==
OpcionModo::MEJOR_RESULTADO){
            if(flag_jugador_1 == 0 || flag_jugador_2 == 0){
                if (inputs.Boton_1 && flag_jugador_1 == 0) {
                    _tiempoReaccion_1 = Tiempo - _tickInicioDisparo;
                    _puntuacion.registrarRonda(_tiempoReaccion_1,
_confModo, 1);

                    _redibujar = true;
                    flag_jugador_1 = 1;
                    // --- NUEVO: feedback sonoro corto ---
                    Buzzer_Beep(50); // Pitido muy corto de 50ms
                }
                if (inputs.Boton_2 && flag_jugador_2 == 0) {
                    _tiempoReaccion_2 = Tiempo - _tickInicioDisparo;
                    _puntuacion.registrarRonda(_tiempoReaccion_2,
_confModo, 2);

                    _redibujar = true;
                    flag_jugador_2 = 1;
                    // --- NUEVO: feedback sonoro corto ---

```

```

        Buzzer_Beep(50); // Pitido muy corto de 50ms
    }
}
if (flag_jugador_1 == 1 && flag_jugador_2 == 1){
    flag_jugador_1 = false;
    flag_jugador_2 = false;
    _tickInicio = Tiempo;
    _estadoActual = EstadoJuego::ST_MOSTRAR_RESULTADO_RONDA;
    HAL_GPIO_WritePin(LED_ROJO_GPIO_Port, LED_ROJO_Pin,
GPIO_PIN_RESET);
}
}

// MODO PARA 2 JUGADORES CON CUENTA_ATRAS
if (_confJugadores == OpcionJugadores::DOS_JUGADORES && _confModo ==
OpcionModo::CUENTA_ATRAS){
    if (inputs.Boton_1 ) {
        _tiempoReaccion_1 = Tiempo - _tickInicioDisparo;
        _puntuacion.registrarRonda(_tiempoReaccion_1, _confModo, 1);
        _redibujar = true;
        _tickInicio = Tiempo;
        _estadoActual = EstadoJuego::ST_MOSTRAR_RESULTADO_RONDA;
        HAL_GPIO_WritePin(LED_ROJO_GPIO_Port, LED_ROJO_Pin,
GPIO_PIN_RESET);

        // --- NUEVO: feedback sonoro corto ---
        Buzzer_Beep(50); // Pitido muy corto de 50ms
    }
    else if (inputs.Boton_2 ) {
        _tiempoReaccion_2 = Tiempo - _tickInicioDisparo;
        _puntuacion.registrarRonda(_tiempoReaccion_2, _confModo, 2);
        _redibujar = true;
        _tickInicio = Tiempo;
        _estadoActual = EstadoJuego::ST_MOSTRAR_RESULTADO_RONDA;
        HAL_GPIO_WritePin(LED_ROJO_GPIO_Port, LED_ROJO_Pin,
GPIO_PIN_RESET);

        // --- NUEVO: feedback sonoro corto ---
        Buzzer_Beep(50); // Pitido muy corto de 50ms
    }
}
break;
case EstadoJuego::ST_MOSTRAR_RESULTADO_RONDA:
    // Aumentamos a 3 segundos para asegurar que el usuario vea el tiempo
    if (Tiempo - _tickInicio >= 3000) {
        _redibujar = true;
        if (_puntuacion.getRondaActual() > 3) {
            _estadoActual = EstadoJuego::ST_JUEGO_FIN;
        } else {
            _estadoActual = EstadoJuego::ST_JUEGO_INICIO;
        }
    }
}
break;
case EstadoJuego::ST_JUEGO_FIN:

```

```

        if (inputs.btn_PA0_Hold) {
            resetearJuegoCompleto();
            _solicitudSalidaMenu = true;
            _redibujar = true;
        }
        else if (inputs.btn_PA0_Click) {
            _puntuacion.resetear();
            _estadoActual = EstadoJuego::ST_JUEGO_INICIO;
            _solicitudSalidaMenu = false;
            _redibujar = true;
        }
        break;
    }
    // 3. LLAMADA CRÍTICA A VISUALIZAR
    visualizar();
}

uint32_t Logica_Juego::obtenerTiempoEspera() {
    // Implementación basada en la dificultad seleccionada
    uint32_t div_dif = (_confDificultad == OpcionDificultad::DIFICIL) ? 2 : 1;
    // Genera un tiempo aleatorio entre 1s y 3s (ajustado por dificultad)
    return (rand() % (2000 / div_dif)) + 1000;
}

void Logica_Juego::configurar(OpcionJugadores jug, OpcionModo modo,
OpcionDificultad dif) {
    _confJugadores = jug;
    _confModo = modo;
    _confDificultad = dif;
}

void Logica_Juego::resetearJuegoCompleto() {
    _puntuacion.resetear();
    _estadoActual = EstadoJuego::ST_JUEGO_APAGADO;
    _tickInicio = 0;
    _tiempoAleatorio = 0;
    _tiempoReaccion_1 = 0;
    _tiempoReaccion_2 = 0;
    HAL_GPIO_WritePin(LED_AZUL_GPIO_Port, LED_AZUL_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LED_ROJO_GPIO_Port, LED_ROJO_Pin, GPIO_PIN_RESET);
}

void Logica_Juego::Buzzer_Beep(uint16_t duracion) {
    HAL_GPIO_WritePin(BUZZER_GPIO_Port, BUZZER_Pin, GPIO_PIN_SET);
    HAL_Delay(duracion);
    HAL_GPIO_WritePin(BUZZER_GPIO_Port, BUZZER_Pin, GPIO_PIN_RESET);
}

void Logica_Juego::visualizar() {
    if (_estadoActual == EstadoJuego::ST_JUEGO_APAGADO) return;
    if (_redibujar) {
        _dibujante.dibujarEstado(_estadoActual, _puntuacion, _confJugadores);
        _redibujar = false;
    }
    if (_estadoActual == EstadoJuego::ST_JUEGO_INICIO) {
        _pantalla->setEscala(1);
    }
}

```

```

    _pantalla->Escribir(40, 180, "PULSA PARA EMPEZAR", 0x07E0, 0, 0);
}
if (_estadoActual == EstadoJuego::ST_JUEGO_RONDA) {
    _pantalla->setEscala(7);
    _pantalla->Escribir(50, 90, "YA!", 0xFFFF, 0, 0);
}
if (_estadoActual == EstadoJuego::ST_MOSTRAR_RESULTADO_RONDA) {
    char buf[32];
    _pantalla->setEscala(4);
    sprintf(buf, "%lu ms", _puntuacion.getUltimoTiempo());
    _pantalla->Escribir(20, 80, buf, 0x07E0, 0, 0);
}
}
void Logica_Juego::iniciarRonda() {
    // Aquí podríamos inicializar contadores o recursos para una nueva ronda.
    // Por ahora se gestiona en actualizar() al pasar a EstadoJuego::ACCION.
}

```

3.2.1. 4. Estilo_pantalla.cpp

Este bloque funciona como una capa de abstracción de alto nivel para el dibujo de gráficos. Mientras que el driver se encarga de la comunicación física, Estilo_Pantalla se encarga de traducir caracteres ASCII y cadenas de texto en mapas de bits visibles en la pantalla.

Para ello, hemos implementado un mapa de bloques de píxeles en una matriz de 8×8 bits. El bloque recorre esta matriz y, mediante operaciones de desplazamiento de bits, determina qué píxeles deben encenderse para formar cada letra. Además, para darle al juego una estética "arcade" o "retro", hemos creado una función de escritura (Escribir) que dibuja el texto en tres capas superpuestas (sombra inferior, sombra derecha y frente), generando un efecto de profundidad o 3D sin necesidad de procesadores gráficos complejos.

CÓDIGO

```

#include "ESTILO_PANTALLA.h"
// --- Fuente Bitmap 8x8 CORREGIDA ---
const uint8_t fuente[40][8] = {
    {0,0,0,0,0,0,0,0}, // 0: Espacio

```

```

// A, B, C, D, E
{0x18,0x3C,0x66,0x66,0x7E,0x66,0x66,0x00},
{0xFC,0x66,0x66,0x7C,0x66,0x66,0xFC,0x00},
{0x3C,0x66,0x60,0x60,0x60,0x66,0x3C,0x00},
{0xF8,0x6C,0x66,0x66,0x66,0x6C,0xF8,0x00},
{0x7E,0x60,0x60,0x78,0x60,0x60,0x7E,0x00},
// F, G, H, I, J
{0x7E,0x60,0x60,0x78,0x60,0x60,0x60,0x00},
{0x3C,0x66,0x60,0x6E,0x66,0x66,0x3C,0x00},
{0x66,0x66,0x66,0x7E,0x66,0x66,0x66,0x00},
{0x3C,0x18,0x18,0x18,0x18,0x18,0x3C,0x00},
{0x1E,0x0C,0x0C,0x0C,0xCC,0xCC,0x78,0x00},
// K, L (Arreglada), M, N, O
{0xE6,0x66,0x6C,0x78,0x6C,0x66,0xE6,0x00},
{0xF0,0x60,0x60,0x60,0x60,0x60,0xFE,0x00},
{0xC6,0xEE,0xFE,0xD6,0xC6,0xC6,0xC6,0x00},
{0xC6,0xE6,0xF6,0xDE,0xCE,0xC6,0xC6,0x00},
{0x3C,0x66,0x66,0x66,0x66,0x66,0x3C,0x00},
// P, Q, R, S, T
{0xFC,0x66,0x66,0x7C,0x60,0x60,0xF0,0x00},
{0x7C,0x66,0x66,0x7C,0x6C,0x66,0x66,0x00},
{0xFC,0x66,0x66,0x7C,0x6C,0x66,0xE6,0x00},
{0x3C,0x66,0x60,0x3C,0x06,0x66,0x3C,0x00},
{0x7E,0x18,0x18,0x18,0x18,0x18,0x18,0x00},
// U, V, W, X, Y, Z
{0x66,0x66,0x66,0x66,0x66,0x66,0x3C,0x00},
{0x66,0x66,0x66,0x66,0x66,0x3C,0x18,0x00},
{0xC6,0xC6,0xD6,0xFE,0xEE,0xC6,0xC6,0x00},
{0xC3,0x66,0x3C,0x18,0x3C,0x66,0xC3,0x00},
{0x66,0x66,0x66,0x3C,0x18,0x18,0x18,0x00},
{0xFE,0x06,0x0C,0x18,0x30,0x60,0xFE,0x00},
// 0-9 (Indices 27-36)
{0x3C,0x66,0x6E,0x76,0x66,0x66,0x3C,0x00},
{0x18,0x18,0x38,0x18,0x18,0x18,0x7E,0x00},
{0x3C,0x66,0x06,0x0C,0x30,0x60,0x7E,0x00},
{0x3C,0x66,0x06,0x1C,0x06,0x66,0x3C,0x00},
{0x0C,0x1C,0x3C,0x6C,0xCC,0xFE,0x0C,0x00},
{0x7E,0x60,0x7C,0x06,0x06,0x66,0x3C,0x00},
{0x3C,0x66,0x60,0x7C,0x66,0x66,0x3C,0x00},
{0x7E,0x06,0x0C,0x18,0x30,0x30,0x30,0x00},
{0x3C,0x66,0x3C,0x66,0x66,0x66,0x3C,0x00},
{0x3C,0x66,0x66,0x3E,0x06,0x66,0x3C,0x00}
};
/*
Bits (0-7)      Hex
Fila 0: ...■...  0x18
Fila 1: ..■■■■..  0x3C
Fila 2: .■...■.  0x66
Fila 3: .■...■.  0x66
Fila 4: .■■■■■.  0x7E
Fila 5: .■...■.  0x66

```



```

Fila 6: .■■■■■■. 0x66
Fila 7: ..... 0x00 */
int getIndice(char c) {
    // función auxiliar: convierte minúsculas a mayúsculas automáticamente
    restando 32
    if (c >= 'a' && c <= 'z') {
        c -= 32;
    }
    if (c == ' ') return 0;
    if (c == ':' || c == '.') return 0; // Tratar puntos y dos puntos como
    espacio (o añadir símbolo)
    if (c >= 'A' && c <= 'Z') return c - 'A' + 1;
    if (c >= '0' && c <= '9') return c - '0' + 27;
    return 0; // Default espacio
}
Estilo_Pantalla::Estilo_Pantalla(Driver_Pantalla* tft, uint8_t escala)
: _tft(tft), _escala(escala) {}
void Estilo_Pantalla::rellenarPantalla(uint16_t color) {
    _tft->rellenarPantalla(color);
}
void Estilo_Pantalla::setEscala(uint8_t nuevaEscala) {
    _escala = nuevaEscala;
}
// PARA ENTENDER ESTO:
/*la funcion dibujar letra se dedica simplemente a dibujar con pegotes de
pixeles (bloques) los
* bits que el mapa de bits de la letra char que tenemos normalizada en esa
megamatriz en la
* posicion y color que se le pasa por parametros
*
* Este pegote de pixels lo ejecuta la funcion dibujar bloque del driver, que es
el que trabaja
* físicamente con la OLED directamente
*
* */
void Estilo_Pantalla::dibujarLetraBase(int x, int y, char c, uint16_t color) {
    int idx = getIndice(c);
    const uint8_t* bitmap = fuente[idx];
    for (int fila = 0; fila < 8; fila++) {
        for (int col = 0; col < 8; col++) {
            if (bitmap[fila] & (1 << (7 - col))) {
                _tft->dibujarBloque(x + (col * _escala), y + (fila * _escala),
color, _escala);
            }
        }
    }
}
// ESTA FUNCION ESCRIBIR se puede considerar la de ALTO NIVEL
/* Le pasamos directamente la posición y texto completo y datos del estilo 3D
retro y se encarga
de llamar a las funciones que dibujan:

```

A un nivel más bajo tenemos dibujar letras, que se encarga de situar una plantilla sobre el lienzo, una máscara de bits normalizado por letra, quien ordena al dibujar bloque (esa función que dibuja pegotes de píxeles sobre el lienzo) en función de los "1" de la máscara de bits.

NOTA: esta función está pensada para escribir una palabra en una altura Y, no un texto largo.

Si es demasiado larga la frase se saldrá por la derecha de la pantalla.

No hay una lógica pensada para continuar escribiendo debajo

*/

```
void Estilo_Pantalla::Escribir(int x, int y, const char* texto,
                                uint16_t colorFrente, uint16_t colorSombraDer,
                                uint16_t colorSombraAbajo)
{
    int cursorX = x;
    int len = strlen(texto);
    for (int i = 0; i < len; i++) {
        char c = texto[i];
        // 1. SOMBRA ABAJO
        if (colorSombraAbajo != 0) // Pequeña optimización: no pintar si es
            negro/transparente
            dibujarLetraBase(cursorX, y + _escala, c, colorSombraAbajo);
        // 2. SOMBRA DERECHA
        if (colorSombraDer != 0)
            dibujarLetraBase(cursorX + _escala, y, c, colorSombraDer);
        // 3. FRENTE
        dibujarLetraBase(cursorX, y, c, colorFrente);
        cursorX += (9 * _escala); // espacios entre letras * la escala
    }
}
```

3.2.1. 5. Driver_pantalla.cpp

```
#include "Driver_Pantalla.h"
```

```
//De cara a no tener to.do en el main se ha creado este código.
```

```
// Es un "DRIVER" que gestiona to.do lo que es la comunicación y control físico
// del TFT ST7789, la pantalla OLED que hemos implementado.
```

```
// DC -> PB1, RES -> PB0
```

```
#define DC_CMD() HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_RESET)
```

```
#define DC_DATA() HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_SET)
```

```
#define RST_LOW() HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET)
```

```
#define RST_HIGH() HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET)
```

```
// --- Comandos ST7789 ---
```

```
#define ST7789_SWRESET 0x01 // Reinicia el chip (soft-reset).
```

```
#define ST7789_SLPOUT 0x11 // Saca a la pantalla del modo sueño para que pueda
trabajar.
```

```
#define ST7789_COLMOD 0x3A // Calidad de color: decide cuántos bits usas por
píxel (ej. 16-bit color).
```

```
#define ST7789_MADCTL 0x36 // Controla la rotación (horizontal/vertical) y el
orden de colores (RGB vs BGR).
```

```
#define ST7789_CASET 0x2A // Dónde pintar (ancho): define desde qué columna
```

hasta cuál vamos a dibujar.

```
#define ST7789_RASET 0x2B // Dónde pintar (alto): define desde qué fila hasta  
cuál vamos a dibujar.
```

```
#define ST7789_RAMWR 0x2C // Prepara la memoria para recibir los datos de los  
píxeles que vas a mandar.
```

```
#define ST7789_DISPON 0x29 // Enciende el panel para que muestre la imagen  
(sino se queda negra).
```

```
#define ST7789_INVON 0x21 // Invierte los colores (lo blanco se ve negro y  
viceversa).
```

```
#define ST7789_NORON 0x13 // Quita efectos raros o zoom y usa la pantalla  
completa estándar.
```

```
Driver_Pantalla::Driver_Pantalla(SPI_HandleTypeDef* hspi, uint16_t ancho,  
uint16_t alto)
```

```
    : _hspi(hspi), _width(ancho), _height(alto) {}
```

```
void Driver_Pantalla::enviarComando(uint8_t cmd) {
```

```
    DC_CMD();
```

```
    HAL_SPI_Transmit(_hspi, &cmd, 1, HAL_MAX_DELAY);
```

```
}
```

```
// cada píxel de la pantalla necesita 2 bytes de información para saber de qué  
color pintarse.
```

```
void Driver_Pantalla::enviarDatos(uint8_t* data, size_t len) {
```

```
    DC_DATA();
```

```
    HAL_SPI_Transmit(_hspi, data, len, HAL_MAX_DELAY);
```

```
}
```

```
void Driver_Pantalla::inicializar() {
```

```
    RST_LOW(); HAL_Delay(20);
```

```
    RST_HIGH(); HAL_Delay(120);
```

```
    enviarComando(ST7789_SWRESET); HAL_Delay(150);
```

```
    enviarComando(ST7789_SLPOUT); HAL_Delay(120);
```

```
    // La lógica es: (1)(Comando) -> (2)(Dato)
```

```
    // PASO 1: Avisamos de QUÉ vamos a configurar
```

```
    // PASO 2: Enviamos el valor de esa configuración
```

```
    enviarComando(ST7789_COLMOD);
```

```
    uint8_t colorMode = 0x55; // trabajar en colores 16-bit
```

```
    enviarDatos(&colorMode, 1);
```

```
    enviarComando(ST7789_MADCTL);
```

```
    uint8_t madctl = 0x00;
```

```
    enviarDatos(&madctl, 1);
```

```
    enviarComando(ST7789_INVON);
```

```
    enviarComando(ST7789_NORON);
```

```
    enviarComando(ST7789_DISPON);
```

```
    HAL_Delay(120);
```

```
}
```

```
void Driver_Pantalla::setVentana(uint16_t x0, uint16_t y0, uint16_t x1, uint16_t  
y1) {
```

```
    uint8_t data[4];
```

```
    // CONSTRUIMOS LA MATRIZ SOBRE LA QUE TRABAJAREMOS con la TFT st7789
```

```
    // definicion de las columnas, eje X
```

```
        enviarComando(ST7789_CASET);
```

```
        // Como x0 es de 16 bits (ej. 240), y SPI envía de 8 en 8, hay que  
partirlo:
```

```

    data[0] = x0 >> 8;    // byte alto, los 8 bits de la izquierda
    data[1] = x0 & 0xFF; // byte bajo, los 8 bits de la derecha
    data[2] = x1 >> 8;    // fin de X, byte alto
    data[3] = x1 & 0xFF; // fin de X, byte bajo
    enviarDatos(data, 4);
    // definimos las filas, eje Y
    enviarComando(ST7789_RASET);
    data[0] = y0 >> 8; data[1] = y0 & 0xFF; // inicio Y
    data[2] = y1 >> 8; data[3] = y1 & 0xFF; // fin Y
    enviarDatos(data, 4);
    enviarComando(ST7789_RAMWR);
}
//DIBUJAR LA PANTALLA
//Recuerda que cada píxel de la pantalla necesita 2 bytes de información para
saber de qué color pintarse.
// USAMOS 16 bits para mostrar lo que deberian ser 24:
// 16 bits que son 5 bits Rojo, 6 bits Verde, 5 bits Azul en RGB565
// en lugar de 24 bits (RGB888): 8 bits Rojo, 8 bits Verde, 8 bits Azul
void Driver_Pantalla::rellenarPantalla(uint16_t color) {
    setVentana(0, 0, _width - 1, _height - 1);
    //LA VARIABLE COLOR YA ESTABA DEFINIDA ANTES, solo divido en 2 conjuntos de 8
bits el mensaje que envio
    uint8_t hi = color >> 8;
    uint8_t lo = color & 0xFF;
    // preparamos un paquete (buffer) donde caben 64 bytes. Como cada píxel ocupa
2 bytes, caben 32 píxeles
    uint8_t buffer[64];
    // llenamos el paquete de (Hi, Lo), (Hi, Lo), (Hi, Lo)...
    for(int i=0; i<64; i+=2) { buffer[i] = hi; buffer[i+1] = lo; }
    DC_DATA();
    uint32_t pixels = _width * _height;
    // Enviamos el mismo buffer (pq es el mismo color siempre) en bloques para no
saturar la RAM
    for(uint32_t i = 0; i < pixels; i += 32) {
        HAL_SPI_Transmit(_hspi, buffer, 64, HAL_MAX_DELAY);
    }
}
void Driver_Pantalla::dibujarBloque(uint16_t x, uint16_t y, uint16_t color,
uint8_t tamano) {
    if((x + tamano) > _width || (y + tamano) > _height) return;
    setVentana(x, y, x + tamano - 1, y + tamano - 1);
    //LA VARIABLE COLOR YA ESTABA DEFINIDA ANTES, solo divido en 2 conjuntos de 8
bits el mensaje que envio
    uint8_t hi = color >> 8;
    uint8_t lo = color & 0xFF;
    // Buffer dinámico según el tamaño del bloque (letra escalada)
    int totalBytes = tamano * tamano * 2;
    uint8_t tempBuf[512]; // Buffer estático para evitar malloc/free constante
    for(int i=0; i<totalBytes && i<512; i+=2){
        tempBuf[i] = hi;
        tempBuf[i+1] = lo;
    }
}

```

```

    }
    DC_DATA();
    HAL_SPI_Transmit(_hspi, tempBuf, totalBytes, HAL_MAX_DELAY);
}

```

3.2.1. 6. Driver_pantalla.cpp

Este bloque constituye el nivel más bajo de software antes de llegar al hardware. Actúa como un "controlador" o driver dedicado para la pantalla TFT ST7789 que hemos implementado. Su responsabilidad es gestionar la comunicación a través del protocolo SPI, enviando tanto los comandos de configuración como los datos de color de cada píxel.

El driver se encarga de la inicialización de la pantalla (enviando la secuencia de comandos necesaria para despertar el chip y configurar el formato de color a 16 bits), así como de definir las "ventanas" de dibujo, es decir bloques de pixeles con los que pintamos sobre la OLED.

Tras prueba y error, optamos por optimizar el rendimiento evitando saturar la memoria RAM del microcontrolador, implementando el envío de datos mediante buffers dinámicos, lo que permite pintar áreas grandes o bloques de color de manera eficiente sin enviar píxel a píxel individualmente.

CÓDIGO

```

#include "Driver_Pantalla.h"

// Macros para el control de pines (Data/Command y Reset)
#define DC_CMD()  HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_RESET)
#define DC_DATA() HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_SET)
#define RST_LOW() HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET)
#define RST_HIGH() HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET)

// --- Comandos ST7789 ---
#define ST7789_SWRESET 0x01
#define ST7789_SLPOUT  0x11
#define ST7789_COLMOD  0x3A
#define ST7789_MADCTL   0x36
#define ST7789_CASET    0x2A
#define ST7789_RASET    0x2B
#define ST7789_RAMWR    0x2C
#define ST7789_DISPON   0x29
#define ST7789_INVON    0x21
#define ST7789_NORON    0x13

```

```

Driver_Pantalla::Driver_Pantalla(SPI_HandleTypeDef* hspi, uint16_t ancho,
uint16_t alto)
    : _hspi(hspi), _width(ancho), _height(alto) {}

void Driver_Pantalla::enviarComando(uint8_t cmd) {
    DC_CMD();
    HAL_SPI_Transmit(_hspi, &cmd, 1, HAL_MAX_DELAY);
}

void Driver_Pantalla::enviarDatos(uint8_t* data, size_t len) {
    DC_DATA();
    HAL_SPI_Transmit(_hspi, data, len, HAL_MAX_DELAY);
}

void Driver_Pantalla::inicializar() {
    RST_LOW(); HAL_Delay(20);
    RST_HIGH(); HAL_Delay(120);

    // Secuencia de inicialización del ST7789
    enviarComando(ST7789_SWRESET); HAL_Delay(150);
    enviarComando(ST7789_SLPOUT); HAL_Delay(120);

    enviarComando(ST7789_COLMOD);
    uint8_t colorMode = 0x55; // 16-bit color
    enviarDatos(&colorMode, 1);

    enviarComando(ST7789_MADCTL);
    uint8_t madctl = 0x00;
    enviarDatos(&madctl, 1);

    enviarComando(ST7789_INVON);
    enviarComando(ST7789_NORON);
    enviarComando(ST7789_DISPON);
    HAL_Delay(120);
}

// Define el área de la pantalla donde se escribirá a continuación
void Driver_Pantalla::setVentana(uint16_t x0, uint16_t y0, uint16_t x1, uint16_t
y1) {
    uint8_t data[4];

    enviarComando(ST7789_CASET); // Columnas
    data[0] = x0 >> 8; data[1] = x0 & 0xFF;
    data[2] = x1 >> 8; data[3] = x1 & 0xFF;
    enviarDatos(data, 4);

    enviarComando(ST7789_RASET); // Filas
    data[0] = y0 >> 8; data[1] = y0 & 0xFF;
    data[2] = y1 >> 8; data[3] = y1 & 0xFF;
    enviarDatos(data, 4);
}

```

```

    enviarComando(ST7789_RAMWR); // Listo para escribir RAM
}

void Driver_Pantalla::rellenarPantalla(uint16_t color) {
    setVentana(0, 0, _width - 1, _height - 1);
    uint8_t hi = color >> 8;
    uint8_t lo = color & 0xFF;

    uint8_t buffer[64];
    for(int i=0; i<64; i+=2) { buffer[i] = hi; buffer[i+1] = lo; }

    DC_DATA();
    uint32_t pixels = _width * _height;
    // Envío por bloques para eficiencia
    for(uint32_t i = 0; i < pixels; i += 32) {
        HAL_SPI_Transmit(_hspi, buffer, 64, HAL_MAX_DELAY);
    }
}

void Driver_Pantalla::dibujarBloque(uint16_t x, uint16_t y, uint16_t color,
uint8_t tamano) {
    if((x + tamano) > _width || (y + tamano) > _height) return;

    setVentana(x, y, x + tamano - 1, y + tamano - 1);
    uint8_t hi = color >> 8;
    uint8_t lo = color & 0xFF;

    int totalBytes = tamano * tamano * 2;
    uint8_t tempBuf[512];

    for(int i=0; i<totalBytes && i<512; i+=2){
        tempBuf[i] = hi;
        tempBuf[i+1] = lo;
    }

    DC_DATA();
    HAL_SPI_Transmit(_hspi, tempBuf, totalBytes, HAL_MAX_DELAY);
}

```

3.2.1. 6. Dibujar_juego.cpp

Este módulo es el que conecta la lógica del juego con lo que el usuario ve realmente. Actúa como una especie de intermediario que traduce los estados internos (como

ST_JUEGO_RONDA o ST_JUEGO_FIN) en imágenes comprensibles, mostrando marcadores, textos de aviso o resultados.

Funciona con una sencilla máquina esclava a la máquina de estados Logica_Juego que va siguiendo a la máquina principal. Para no ocasionarle problemas a el dibujante de Maquina_Menu, la pantalla solo se borra y se redibuja cuando hay un cambio de estado real, no en cada ciclo del procesador. Además, aquí es donde convertimos los datos numéricos (como los milisegundos de reacción) en texto legible para enseñarlo en el marcador final.

CÓDIGO

```
#include "Dibujar_Juego.hpp"
#include <stdio.h>
DibujarJuego::DibujarJuego(Estilo_Pantalla* pantalla) : _pantalla(pantalla) {}
void DibujarJuego::dibujarEstado(EstadoJuego estado, const GestorPuntuacion&
puntos, OpcionJugadores nJug) {
    char buf[32];
    // Limpiamos la pantalla SOLO una vez al cambiar de estado
    _pantalla->rellenarPantalla(0x0000);
    switch (estado) {
        case EstadoJuego::ST_JUEGO_INICIO:
            _pantalla->setEscala(4);
            sprintf(buf, "RONDA %d", puntos.getRondaActual());
            _pantalla->Escribir(20, 70, buf, 0xFFFF, 0x001F, 0x001F);
            // El "PULSA PARA EMPEZAR" se omite aquí para que lo dibuje
Logica_Juego::visualizar
            break;
        case EstadoJuego::ST_TIEMPO_PREPARACION:
            _pantalla->rellenarPantalla(0x001F); // Fondo Azul fijo
            _pantalla->setEscala(3);
            _pantalla->Escribir(30, 100, "LISTO?", 0xFFFF, 0, 0);
            break;
        case EstadoJuego::ST_JUEGO_RONDA:
            _pantalla->rellenarPantalla(0xF800); // Fondo Rojo fijo
            break;
        case EstadoJuego::ST_MOSTRAR_RESULTADO_RONDA:
            _pantalla->setEscala(4);
            sprintf(buf, "%lu ms", puntos.getUltimoTiempo());
            _pantalla->Escribir(20, 80, buf, 0x07E0, 0, 0);
            dibujarMarcador(puntos, nJug);
            break;
        case EstadoJuego::ST_JUEGO_FIN:
            _pantalla->setEscala(3);
            // Reducimos el texto o la escala para que no se corte
            _pantalla->Escribir(0, 50, "FIN JUEGO", 0xFFFF, 0xF800, 0);
            // Bajamos el marcador para que no se pegue al título
            dibujarMarcador(puntos, nJug);
            _pantalla->setEscala(1.7);
            // Bajamos las instrucciones al borde inferior
```



```

        _pantalla->Escribir(25, 200, "CLICK: REINTENTAR", 0x07E0, 0, 0);
        _pantalla->Escribir(25, 220, "HOLD: SALIR", 0xF800, 0, 0);
        break;
    default: break;
}
}
}
void DibujarJuego::dibujarMarcador(const GestorPuntuacion& puntos,
OpcionJugadores nJug) {
    char buf[32];
    _pantalla->setEscala(2); // Escala 2 para que se vea mejor que la 1
    sprintf(buf, "J1: %lu PTS", puntos.getPuntosJ1());
    _pantalla->Escribir(20, 130, buf, 0xFFFF, 0, 0);
    if (nJug == OpcionJugadores::DOS_JUGADORES) {
        sprintf(buf, "J2: %lu PTS", puntos.getPuntosJ2());
        _pantalla->Escribir(20, 160, buf, 0xFFFF, 0, 0);
    }
}
}

```

3.2.1. 7. Main.cpp

El bloque main.cpp es el principal bloque de todo el programa. Es el que inicializa al resto de bloques y el que

CÓDIGO

```

#include <string.h>
#include <stdint.h>
#include "main.h"
#include "DRIVER_PANTALLA.h"
#include "ESTILO_PANTALLA.h"
#include "Logica_Juego.h"
#include "Maquina_Menu.h"
#include "Definiciones.h"
#include "Captura_boton.h"
/* Private includes -----*/
/* USER CODE BEGIN Includes */
/* USER CODE END Includes */
/* Private typedef -----*/
/* USER CODE BEGIN PTD */
/* USER CODE END PTD */
/* Private define -----*/
/* USER CODE BEGIN PD */
// MIRAR EL MAIN.H PARA EL TEMA DE LOS BOTONES Y LEDS GPIO DEL JUEGO -> USER
CODE BEGIN Private defines
/* USER CODE END PD */

```

```

/* Private macro -----*/
/* USER CODE BEGIN PM */
/* USER CODE END PM */
/* Private variables -----*/
SPI_HandleTypeDef hspi1;
ADC_HandleTypeDef hadc1;
TIM_HandleTypeDef htim2;
/* USER CODE BEGIN PV */
Driver_Pantalla* miPantalla;
Estilo_Pantalla* graficos;
Logica_Juego* juego;
Maquina_Menu* consola;
volatile bool irq_pa0_pressed = false;
volatile bool Presiona_Jugador_1 = false;
volatile bool Presiona_Jugador_2 = false;
/* USER CODE END PV */
/* Private function prototypes -----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_SPI1_Init(void);
static void MX_ADC1_Init(void);
static void MX_TIM2_Init(void);
/* USER CODE BEGIN PFP */
/* USER CODE END PFP */
/* Private user code -----*/
/* USER CODE BEGIN 0 */
// 1. Interrupción de GPIO (Botones)
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (GPIO_Pin == GPIO_PIN_0) {
        irq_pa0_pressed = true;
    }
    if (GPIO_Pin == GPIO_PIN_6) {           //J1
        Presiona_Jugador_1 = true;
    }
    if (GPIO_Pin == GPIO_PIN_4) {           //J2
        Presiona_Jugador_2 = true;
    }
}
void Buzzer_Beep(uint16_t duracion_ms);
/* USER CODE END 0 */
/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */
    /* USER CODE END 1 */
    /* MCU Configuration-----*/
    /* Reset of all peripherals, Initializes the Flash interface and the Systick.
    */

```

```

HAL_Init();
/* USER CODE BEGIN Init */
/* USER CODE END Init */
/* Configure the system clock */
SystemClock_Config();
/* USER CODE BEGIN SysInit */
/* USER CODE END SysInit */
/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_SPI1_Init();
MX_ADC1_Init();
MX_TIM2_Init();
/* USER CODE BEGIN 2 */

//PARA LA PANTALLA
// pasamos el SPI (&hspi1) y el tamaño (240x240) al driver y lo configuramos
miPantalla = new Driver_Pantalla(&hspi1, 240, 240);
miPantalla->inicializar();
miPantalla->rellenarPantalla(0x0005); // si queremos 0x0000 negro puro.
//Para el dibujo en si, no deberia llamarse ESTILO PANTALLA sino dibujante o
algo asi
graficos = new Estilo_Pantalla(miPantalla, 2); // Fuente tamaño medio
juego = new Logica_Juego(graficos);
consola = new Maquina_Menu(graficos, juego);
Captura_Boton_Init(); // Configura TIM/ADC para la captura de
tiempos
juego->resetearJuegoCompleto(); // Deja el juego en estado inicial/coherente
Entradas inputs = {0};
/*
// Colores para hacer la letra retro
uint16_t COLOR_FRENTE = 0xFFE0; // Amarillo brillante
uint16_t SOMBRA_DER = 0xF800; // Rojo
uint16_t SOMBRA_ABAJO = 0x001F; // Azul
// Sintaxis: Escribir(x, y, "TEXTO", ColorFrente, ColorSombraDer,
ColorSombraAbajo)
textoGlitch->Escribir(50, 75, "GAME", COLOR_FRENTE, SOMBRA_DER,
SOMBRA_ABAJO);
textoGlitch->Escribir(50, 125, "OVER", COLOR_FRENTE, SOMBRA_DER,
SOMBRA_ABAJO);
*/
/* USER CODE END 2 */
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1) {
    memset(&inputs, 0, sizeof(Entradas));
    if (irq_pa0_pressed) {
        // Antirrebote inicial
        uint32_t Tiempo_While = HAL_GetTick();
        while (HAL_GetTick() - Tiempo_While < 40);
        if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET) {
            uint32_t t_inicio = HAL_GetTick();
            // ESPERA SILENCIOSA: No llamamos a actualizar aquí para no

```

```

saltar estados
        while (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET) {
            if ((HAL_GetTick() - t_inicio) > 600) {
                inputs.btn_PA0_Hold = true;
                break;
            }
        }
        // Si soltó antes de 600ms, es un Click
        if (!inputs.btn_PA0_Hold) {
            inputs.btn_PA0_Click = true;
        }
        // ESPERA A SOLTAR DEFINITIVAMENTE: Sin llamar a actualizar
        while (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET);
    }
    irq_pa0_pressed = false;
    __HAL_GPIO_EXTI_CLEAR_IT(GPIO_PIN_0);
    // Solo ahora que hemos soltado y decidido qué era (Click o
Hold),

    // procesamos la lógica UNA VEZ.
    consola->actualizar(inputs);
    // Delay de cortesía para que la pantalla respire tras el
procesamiento
    HAL_Delay(150);
}
//BOTON JUGADOR 1
if (Presiona_Jugador_1) {
    // Antirrebote inicial
    uint32_t Tiempo_While = HAL_GetTick();
    while (HAL_GetTick() - Tiempo_While < 40);
    if (HAL_GPIO_ReadPin(GPIOD, GPIO_PIN_6) == GPIO_PIN_SET) {
        uint32_t t_inicio = HAL_GetTick();
        inputs.Boton_1 = true;
    }
    Presiona_Jugador_1 = false;
    __HAL_GPIO_EXTI_CLEAR_IT(GPIO_PIN_6);
    // Solo ahora que hemos soltado y decidido qué era (Click o
Hold),

    // procesamos la lógica UNA VEZ.
    consola->actualizar(inputs);
    // Delay de cortesía para que la pantalla respire tras el
procesamiento
    HAL_Delay(150);
}
//BOTON JUGADOR 2
if (Presiona_Jugador_2) {
    // Antirrebote inicial
    uint32_t Tiempo_While = HAL_GetTick();
    while (HAL_GetTick() - Tiempo_While < 40);
    if (HAL_GPIO_ReadPin(GPIOD, GPIO_PIN_4) == GPIO_PIN_SET) {
        uint32_t t_inicio = HAL_GetTick();
        inputs.Boton_2 = true;
    }
}

```

```

        }
        Presiona_Jugador_2 = false;
        __HAL_GPIO_EXTI_CLEAR_IT(GPIO_PIN_4);
        // Solo ahora que hemos soltado y decidido qué era (Click o
Hold),

        // procesamos la lógica UNA VEZ.
        consola->actualizar(inputs);
        // Delay de cortesía para que la pantalla respire tras el
procesamiento
        HAL_Delay(150);
    }
    else {
        // Si no hay interrupción, el menú sigue su curso normal
(animaciones, etc.)
        consola->actualizar(inputs);
    }
    // Estabilidad del sistema
    uint32_t Tiempo_While_Loop = HAL_GetTick();
    while (HAL_GetTick() - Tiempo_While_Loop < 10);
}
}
/* USER CODE END 3 */
/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
    /** Configure the main internal regulator output voltage
    */
    __HAL_RCC_PWR_CLK_ENABLE();
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);
    /** Initializes the RCC Oscillators according to the specified parameters
    * in the RCC_OscInitTypeDef structure.
    */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSISState = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
    RCC_OscInitStruct.PLL.PLLM = 8;
    RCC_OscInitStruct.PLL.PLLN = 100;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV4;
    RCC_OscInitStruct.PLL.PLLQ = 8;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }
    /** Initializes the CPU, AHB and APB buses clocks

```

```

*/
RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYCLK
                             |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;
if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_1) != HAL_OK)
{
    Error_Handler();
}
}
/**
 * @brief SPI1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_SPI1_Init(void)
{
    /* USER CODE BEGIN SPI1_Init 0 */
    /* USER CODE END SPI1_Init 0 */
    /* USER CODE BEGIN SPI1_Init 1 */
    /* USER CODE END SPI1_Init 1 */
    /* SPI1 parameter configuration*/
    hspi1.Instance = SPI1;
    hspi1.Init.Mode = SPI_MODE_MASTER;
    hspi1.Init.Direction = SPI_DIRECTION_2LINES;
    hspi1.Init.DataSize = SPI_DATASIZE_8BIT;
    hspi1.Init.CLKPolarity = SPI_POLARITY_HIGH; //ANTES ESTABA LOW
    hspi1.Init.CLKPhase = SPI_PHASE_2EDGE; //ANTES ESTABA 1
    hspi1.Init.NSS = SPI_NSS_SOFT;
    hspi1.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_2;
    hspi1.Init.FirstBit = SPI_FIRSTBIT_MSB;
    hspi1.Init.TIMode = SPI_TIMODE_DISABLE;
    hspi1.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE;
    hspi1.Init.CRCPolynomial = 10;
    if (HAL_SPI_Init(&hspi1) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN SPI1_Init 2 */
    /* USER CODE END SPI1_Init 2 */
}
/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

```

```

/* USER CODE BEGIN MX_GPIO_Init_1 */
/* USER CODE END MX_GPIO_Init_1 */
/* GPIO Ports Clock Enable */
__HAL_RCC_GPIOC_CLK_ENABLE();
__HAL_RCC_GPIOA_CLK_ENABLE();
__HAL_RCC_GPIOB_CLK_ENABLE();
__HAL_RCC_GPIOD_CLK_ENABLE();
/*Configure GPIO pin Output Level */
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0|GPIO_PIN_1|BUZZER_Pin, GPIO_PIN_RESET);
/*Configure GPIO pin Output Level */
HAL_GPIO_WritePin(GPIOD, LED_ROJO_Pin|LED_AZUL_Pin, GPIO_PIN_RESET);
/*Configure GPIO pin : BOTON_AZUL_Pin */
GPIO_InitStruct.Pin = BOTON_AZUL_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
GPIO_InitStruct.Pull = GPIO_PULLDOWN;
HAL_GPIO_Init(BOTON_AZUL_GPIO_Port, &GPIO_InitStruct);
/*Configure GPIO pins : PB0 PB1 BUZZER_Pin */
GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1|BUZZER_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
/*Configure GPIO pins : LED_ROJO_Pin LED_AZUL_Pin */
GPIO_InitStruct.Pin = LED_ROJO_Pin|LED_AZUL_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);
/*Configure GPIO pins : PA10 PA11 */
GPIO_InitStruct.Pin = GPIO_PIN_10|GPIO_PIN_11;
GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
/*Configure GPIO pins : BTN_J2_Pin BTON_J1_Pin */
GPIO_InitStruct.Pin = BTN_J2_Pin|BTON_J1_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
GPIO_InitStruct.Pull = GPIO_PULLDOWN;
HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);
/* EXTI interrupt init*/
HAL_NVIC_SetPriority(EXTI0_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(EXTI0_IRQn);
HAL_NVIC_SetPriority(EXTI4_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(EXTI4_IRQn);
HAL_NVIC_SetPriority(EXTI9_5_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(EXTI9_5_IRQn);
HAL_NVIC_SetPriority(EXTI15_10_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
/* USER CODE BEGIN MX_GPIO_Init_2 */
/* USER CODE END MX_GPIO_Init_2 */
}
static void MX_ADC1_Init(void)

```

```

{
    /* USER CODE BEGIN ADC1_Init 0 */
    /* USER CODE END ADC1_Init 0 */
    ADC_ChannelConfTypeDef sConfig = {0};
    /* USER CODE BEGIN ADC1_Init 1 */
    /* USER CODE END ADC1_Init 1 */
    /** Configure the global features of the ADC (Clock, Resolution, Data Alignment
    and number of conversion)
    */
    hadc1.Instance = ADC1;
    hadc1.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV2;
    hadc1.Init.Resolution = ADC_RESOLUTION_12B;
    hadc1.Init.ScanConvMode = DISABLE;
    hadc1.Init.ContinuousConvMode = ENABLE;
    hadc1.Init.DiscontinuousConvMode = DISABLE;
    hadc1.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;
    hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;
    hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
    hadc1.Init.NbrOfConversion = 1;
    hadc1.Init.DMAContinuousRequests = DISABLE;
    hadc1.Init.EOCSelection = ADC_EOC_SINGLE_CONV;
    if (HAL_ADC_Init(&hadc1) != HAL_OK)
    {
        Error_Handler();
    }
    /** Configure for the selected ADC regular channel its corresponding rank in
    the sequencer and its sample time.
    */
    sConfig.Channel = ADC_CHANNEL_10;
    sConfig.Rank = 1;
    sConfig.SamplingTime = ADC_SAMPLETIME_3CYCLES;
    if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN ADC1_Init 2 */
    /* USER CODE END ADC1_Init 2 */
}

static void MX_TIM2_Init(void)
{
    /* USER CODE BEGIN TIM2_Init 0 */
    /* USER CODE END TIM2_Init 0 */
    TIM_ClockConfigTypeDef sClockSourceConfig = {0};
    TIM_MasterConfigTypeDef sMasterConfig = {0};
    /* USER CODE BEGIN TIM2_Init 1 */
    /* USER CODE END TIM2_Init 1 */
    htim2.Instance = TIM2;
    htim2.Init.Prescaler = 83;
    htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim2.Init.Period = 999;
    htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;

```



```

htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
{
    Error_Handler();
}
sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK)
{
    Error_Handler();
}
sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN TIM2_Init 2 */
/* USER CODE END TIM2_Init 2 */
}
/* USER CODE BEGIN 4 */
void Buzzer_Beep(uint16_t duracion_ms)
{
    HAL_GPIO_WritePin(BUZZER_GPIO_Port, BUZZER_Pin, GPIO_PIN_SET);
    HAL_Delay(duracion_ms);
    HAL_GPIO_WritePin(BUZZER_GPIO_Port, BUZZER_Pin, GPIO_PIN_RESET);
}
/* USER CODE END 4 */
/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 * where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{

```

```

/* USER CODE BEGIN 6 */
/* User can add his own implementation to report the file name and line number,
   ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
/* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */

```

3.2.2. Bloques .h

3.2.2. 1. Captura_boton.h

```

#ifndef INC_CAPTURA_BOTON_H_
#define INC_CAPTURA_BOTON_H_
#include <stdint.h>
void Captura_Boton_Init(void);
void Captura_Boton_Run(void);
// Funciones auxiliares para consultar el resultado de la última captura
// Devuelve el tiempo de reacción medido en milisegundos desde el inicio de la
ronda.
uint32_t Captura_GetTiempoFinal();
// Devuelve el identificador del jugador ganador (1 o 2). Si no se ha registrado
// ninguna pulsación aún, devuelve 0.
uint8_t Captura_GetGanador();
#endif /* INC_CAPTURA_BOTON_H_ */

```

3.2.2. 2. Maquina_menu.h

```

#ifndef MAQUINA_MENU_H
#define MAQUINA_MENU_H
#include "Definiciones.h"
#include "ESTILO_PANTALLA.h"
#include "Logica_Juego.h"
#include "main.h"
class Maquina_Menu {
private:
    Estilo_Pantalla* _pantalla;
    Logica_Juego* _juego;
    EstadoMenu _estadoActual;
    OpcionJugadores _selJugadores;
    OpcionModo _selModo;
    OpcionDificultad _selDificultad;
    uint32_t _tickParpadeo;
    bool _textoVisible;
    bool _inicioDibujado;
    uint32_t _tiempoBloqueo;

```

```

void dibujarPantallaInicio();
void dibujarMenuJugadores();
void dibujarMenuModo();
void dibujarMenuDificultad();
void dibujarCarga();
public:
    Maquina_Menu(Estilo_Pantalla* pantalla, Logica_Juego* juego);
    void actualizar(const Entradas& inputs);
    EstadoMenu getEstado() { return _estadoActual; }
};
#endif

```

3.2.2. 3. Lógica_Juego.h

```

#ifndef LOGICA_JUEGO_H
#define LOGICA_JUEGO_H
#include "ESTILO_PANTALLA.h"
#include "Definiciones.h"
#include "GestorPuntuacion.hpp"
#include "Dibujar_Juego.hpp"
#include "main.h"
class Logica_Juego {
public:
    Logica_Juego(Estilo_Pantalla* pantalla);
    void configurar(OpcionJugadores jug, OpcionModo modo, OpcionDificultad dif);
    void resetearJuegoCompleto();
    void actualizar(const Entradas& inputs);
    EstadoJuego getEstadoActual() const { return _estadoActual; }
    bool getSolicitudSalidaMenu() const { return _solicitudSalidaMenu; }
    void setSolicitudSalidaMenu(bool valor) { _solicitudSalidaMenu = valor; }
private:
    Estilo_Pantalla* _pantalla;
    // Dibujante se encarga de pintar las escenas del juego (pantallas y
    marcador).
    DibujarJuego _dibujante;
    // Gestor de puntuación para llevar el registro de rondas y puntos de cada
    jugador.
    GestorPuntuacion _puntuacion;
    OpcionJugadores _confJugadores;
    OpcionModo _confModo;
    OpcionDificultad _confDificultad;
    bool _solicitudSalidaMenu;
    EstadoJuego _estadoActual;
    bool _redibujar;
    uint32_t _tickInicio;
    uint32_t _tickInicioDisparo;
    uint32_t _tiempoAleatorio;

```

```

uint32_t _tiempoReaccion_1;
uint32_t _tiempoReaccion_2;
//flags para el modo 2 jugadores
bool flag_jugador_1 = 0;
bool flag_jugador_2 = 0;
void Buzzer_Beep(uint16_t duracion);
// Métodos auxiliares
void visualizar();
void iniciarRonda();
uint32_t obtenerTiempoEspera();
};
#endif

```

3.2.2. 4. Estilo_pantalla.h

```

#ifndef ESTILO_PANTALLA_H
#define ESTILO_PANTALLA_H
#include "DRIVER_PANTALLA.h" // Incluimos el driver de hardware que ya tienes
#include <string.h>
class Estilo_Pantalla {
private:
    Driver_Pantalla* _tft; // Puntero al driver de la pantalla
    uint8_t _escala; // Tamaño de la letra
    // Método privado auxiliar para dibujar un caracter simple sin efectos
    void dibujarLetraBase(int x, int y, char c, uint16_t color);
public:
    Estilo_Pantalla(Driver_Pantalla* tft, uint8_t escala);
    void rellenarPantalla(uint16_t color);
    void setEscala(uint8_t nuevaEscala);
    void Escribir(int x, int y, const char* texto,
                  uint16_t colorFrente,
                  uint16_t colorSombraDer,
                  uint16_t colorSombraAbajo);
};
#endif

```

3.2.2. 5. Driver_pantalla.h

```

#ifndef DRIVER_PANTALLA_H
#define DRIVER_PANTALLA_H
#include "main.h"
class Driver_Pantalla {
private:
    SPI_HandleTypeDef* _hspi;
    uint16_t _width;
    uint16_t _height;
    // Métodos privados internos del driver

```

```

void enviarComando(uint8_t cmd);
void enviarDatos(uint8_t* data, size_t len);
void setVentana(uint16_t x0, uint16_t y0, uint16_t x1, uint16_t y1);
public:
    // Constructor
    Driver_Pantalla(SPI_HandleTypeDef* hspi, uint16_t ancho, uint16_t alto);
    // Métodos para el usuario
    void inicializar();
    void rellenarPantalla(uint16_t color);
    // Dibuja un cuadrado de color (bloque) en una posición.
    // Es para dibujar píxeles gordos.
    void dibujarBloque(uint16_t x, uint16_t y, uint16_t color, uint8_t tamano);
};
#endif

```

3.2.2. 6. Dibujar_juego.h

```

#ifndef DIBUJAR_JUEGO_HPP
#define DIBUJAR_JUEGO_HPP
#include "ESTILO_PANTALLA.h"
#include "Definiciones.h"
#include "GestorPuntuacion.hpp"
class DibujarJuego {
public:
    DibujarJuego(Estilo_Pantalla* pantalla);
    void dibujarEstado(EstadoJuego estado, const GestorPuntuacion& puntos,
OpcionJugadores nJug);
private:
    Estilo_Pantalla* _pantalla;
    void dibujarFalso(uint16_t color);
    void dibujarMarcador(const GestorPuntuacion& puntos, OpcionJugadores nJug);
};
#endif

```

3.2.2. 7. Main.h

```

/* Define to prevent recursive inclusion -----*/
#ifndef __MAIN_H
#define __MAIN_H
#ifdef __cplusplus
extern "C" {
#endif
/* Includes -----*/
#include "stm32f4xx_hal.h"
/* Private includes -----*/
/* USER CODE BEGIN Includes */
#include <stdlib.h>
extern volatile uint32_t tiempo_ms;

```

```

/* USER CODE END Includes */
/* Exported types -----*/
/* USER CODE BEGIN ET */
/* USER CODE END ET */
/* Exported constants -----*/
/* USER CODE BEGIN EC */
/* USER CODE END EC */
/* Exported macro -----*/
/* USER CODE BEGIN EM */
/* USER CODE END EM */
void HAL_TIM_MspPostInit(TIM_HandleTypeDef *htim);
/* Exported functions prototypes -----*/
void Error_Handler(void);
/* USER CODE BEGIN EFP */
/* USER CODE END EFP */
/* Private defines -----*/
#define BOTON_AZUL_Pin GPIO_PIN_0
#define BOTON_AZUL_GPIO_Port GPIOA
#define BOTON_AZUL_EXTI_IRQn EXTI0_IRQn
#define BUZZER_Pin GPIO_PIN_14
#define BUZZER_GPIO_Port GPIOB
#define LED_ROJO_Pin GPIO_PIN_14
#define LED_ROJO_GPIO_Port GPIOD
#define LED_AZUL_Pin GPIO_PIN_15
#define LED_AZUL_GPIO_Port GPIOD
#define BTN_J2_Pin GPIO_PIN_4
#define BTN_J2_GPIO_Port GPIOD
#define BTN_J2_EXTI_IRQn EXTI4_IRQn
#define BTON_J1_Pin GPIO_PIN_6
#define BTON_J1_GPIO_Port GPIOD
#define BTON_J1_EXTI_IRQn EXTI9_5_IRQn
/* USER CODE BEGIN Private defines */
/*#define LED_AZUL_GPIO_Port GPIOD //para la
logica del juego
#define LED_AZUL_Pin GPIO_PIN_15
#define LED_ROJO_GPIO_Port GPIOD
#define LED_ROJO_Pin GPIO_PIN_14
#define BOTON_JUGADOR_1_GPIO_Port GPIOD
#define BOTON_JUGADOR_1_Pin GPIO_PIN_6
#define BOTON_JUGADOR_2_GPIO_Port GPIOD
#define BOTON_JUGADOR_2_Pin GPIO_PIN_4*/
/* USER CODE END Private defines */
#ifdef __cplusplus
}
#endif
#endif /* __MAIN_H */

```

3.2.2. 8. GestorPuntuacion.h

Este archivo se encarga de la gestión de la puntuación, en función de la ronda del juego. Su función principal es centralizar el cálculo y almacenamiento de los puntos de cada jugador, manteniendo separada esta responsabilidad de la lógica principal del juego. Se mantienen internamente los puntos de cada jugador, el número de ronda actual y el último tiempo de reacción registrado. Además se incluye un reinicio que pone todos estos valores a cero, para que así cada ronda sea diferente.

Se recibe como entrada el tiempo de reacción, el modo de juego y el jugador ganador. En función del modo seleccionado, el sistema asigna puntos de forma distinta, en el modo de cuenta atrás se otorga una puntuación fija al jugador más rápido, mientras que en el modo de mejor resultado la puntuación depende del tiempo obtenido, haciendo que cuanto más rápida sea tu reacción más puntos obtienes.

```
#ifndef GESTOR_PUNTUACION_HPP
#define GESTOR_PUNTUACION_HPP
#include <stdint.h>
#include "Definiciones.h"
class GestorPuntuacion {
public:
    GestorPuntuacion() { resetear(); }
    void resetear() {
        _puntosJ1 = 0; _puntosJ2 = 0;
        _rondaActual = 1; _ultimoTR = 0;
    }
    void registrarRonda(uint32_t tr, OpcionModo modo, uint8_t ganador) {
        _ultimoTR = tr;
        if (modo == OpcionModo::CUENTA_ATRAS) {
            // Punto fijo al que pulsa primero
            if (ganador == 1) _puntosJ1 += 100;
            else if (ganador == 2) _puntosJ2 += 100;
        }
        else if (modo == OpcionModo::MEJOR_RESULTADO) {
            uint16_t pts = calcularPuntos(tr);
            if (ganador == 1) _puntosJ1 += pts;
            else if (ganador == 2) _puntosJ2 += pts;
        }
        _rondaActual++;
    }
    uint8_t getRonda() const { return _rondaActual; }
    uint32_t getPuntosJ1() const { return _puntosJ1; }
    uint32_t getPuntosJ2() const { return _puntosJ2; }
    uint32_t getUltimoTR() const { return _ultimoTR; }
    uint8_t getRondaActual() const { return getRonda(); }
    uint32_t getUltimoTiempo() const { return getUltimoTR(); }
private:
    uint32_t _puntosJ1, _puntosJ2, _ultimoTR;
    uint8_t _rondaActual;
    OpcionJugadores _confJugadores;
    uint16_t calcularPuntos(uint32_t tr) {
```

```

        if (tr < 200) return 100;
        if (tr < 400) return 75;
        if (tr < 600) return 50;
        if (tr < 1000) return 25;
        return 10;
    }
};
#endif

```

3.2.2. 9. Definiciones.h

Este es un archivo central de definiciones, el objetivo es declarar los tipos de datos comunes que se usan en los distintos módulos y así facilitar la comunicación entre estos. En primer lugar, define los estados del menú mediante una enumeración. Estos estados representan las distintas pantallas y fases por las que pasa el usuario antes y durante el juego, como la selección de jugadores, el modo de juego, la dificultad o la ejecución de la partida.

A continuación, se definen las opciones configurables, como el número de jugadores, el modo de juego y la dificultad. Los valores intermedios denominados "fantasma" se utilizan para gestionar la navegación cíclica entre opciones sin afectar a la funcionalidad final.

Se define la estructura Entradas, donde se agrupan todas las acciones que puede realizar el usuario

Y por último, tenemos los distintos estados que se trabajan en la máquina de estados correspondiente a la lógica del juego, Logica_Juego.

```

#ifndef DEFINICIONES_H
#define DEFINICIONES_H
#include <stdint.h>
#include <stdbool.h>
// Estados de La CONSOLA (Menú Principal)
enum class EstadoMenu {
    APAGADO= 0,
    SEL_JUGADORES, // 1 o 2
    SEL_MODO,      // Cuenta atrás vs Mejor resultado
    SEL_DIFICULTAD, // Facil, Medio, Dificil
    PANTALLA_CARGA, // "Empezar"
    JUGANDO        // Aquí corre La lógica del juego
};
// Opciones seleccionables
enum class OpcionJugadores { UN_JUGADOR, FANTASMA_1, DOS_JUGADORES, FANTASMA_2 };
enum class OpcionModo      { CUENTA_ATRAS, FANTASMA_3, MEJOR_RESULTADO, FANTASMA_4 };
enum class OpcionDificultad{ FACIL, FANTASMA_5, MEDIO, FANTASMA_6,
DIFICIL, FANTASMA_7 };

```



```

// Estructura para pasarle los botones a la lógica sin ensuciar con HAL_GPIO
struct Entradas {
    bool btn_PA0_Click;    // Click simple (Confirmar / Encender)
    bool btn_PA0_Hold;     // Mantener (Para encendido inicial si quieres)
    bool btn_Atras;       // Volver al menú anterior
    bool btn_Salir;        // Reset global (vuelve a APAGADO)
    bool btn_ReiniciarJuego; // Reinicia solo el juego (estando dentro)
    bool Boton_1;          // Click J1
    bool Boton_2;          // Click J2
    // Joystick (simulado por ahora como flags)
    bool joy_Arriba;
    bool joy_Abajo;
};

enum class EstadoJuego {
    ST_JUEGO_APAGADO,
    ST_JUEGO_INICIO,
    ST_TIEMPO_PREPARACION,
    ST_TIEMPO_DISPARO,
    ST_JUEGO_RONDA,
    ST_MOSTRAR_RESULTADO_RONDA,
    ST_JUEGO_FIN
};
#endif

```

4. Github

https://github.com/lorenzohidalgoanero/Trabajo_micros

5. Conclusión

Durante la realización de este trabajo hemos logrado aprender en profundidad el funcionamiento de un microcontrolador STM32F411. No solo eso, si no que hemos mejorado nuestras capacidades de diseño y estructuración debido a que nos hemos visto forzados a ello para poder llevar a cabo el proyecto que teníamos en mente.

El proyecto ha logrado cumplir con cada una de los objetivos propuestos. Una de las cualidades de este proyecto que más nos ha demandado ha sido la modularidad de los bloques. Esto sumado a que a mitad del desarrollo nos encontramos con que una parte del equipo había programado en C++ la pantalla OLED, dado que era más fácil trabajar con ella en C++ debido a su complejidad. Esto significó que tuvimos que adaptar a C++ aproximadamente la mitad del código que estaba escrito en C, lo cuál nos llevó varios días y nos creo muchos errores a lo largo de la adaptación.

En cuanto a mejoras a añadir, nos hubiera gustado haber terminado de implementar los modos de dificultad, los cuales debido a la falta de tiempo no nos dió tiempo a depurar todos los errores y al final los modos no afectan a la jugabilidad. Otra cosa que hubiéramos cambiado en caso de tener más tiempo es un pequeño error en el que los botones se detectan 2 veces cada vez que se pulsan. Esto solo afectaba al menú de selección y lo solucionamos añadiendo estados fantasma entre medias de los estados visibles. Por último, nos hubiera gustado mejora es el hardware utilizado, nos hubiera gustado hacer una maqueta en condiciones para hacer el juego más disfrutable.

Salvo el selector de dificultad los otros cambios no afectan a la jugabilidad de "Reflejos al Límite". Y la mejora de la dificultad es una mejora añadida que en caso de no lograrse no significa que el proyecto ha fracasado.

Por último, viendo que se han cumplido todos los objetivos y que hemos logrado obtener un juego "Reflejos al Límite" funcional, podemos dar como concluido el proyecto de manera satisfactoria.